seq2seq-and-attention

深度学习

学　　号 2354369

姓　　名 陈宇轩

专　　业 基础学科拔尖基地（计算机）

授课老师 程大伟

目录

一. 模型解释.....3
1.1 Seq2Seq.....3
1.2 Attention.....3
二. 实验过程与结果.....4
2.1 代码补全.....4
2.2 实验结果.....6

一. 模型解释

1.1 Seq2Seq

Seq2Seq(Sequence to Sequence)模型是一种专门用于处理序列到序列转换问题的神经网络架构。模型的核心思想是将一个可变长度的输入序列映射到另一个可变长度的输出序列。模型采用编码器-解码器架构：编码器负责将输入序列编码成一个固定长度的向量表示,解码器则根据这个向量表示逐步生成输出序列。这种架构的关键创新在于它能够处理输入和输出序列长度不一致的情况,这是在许多实际应用中常见的场景。例如,在机器翻译中,源语言和目标语言的句子长度往往不同;在文本摘要中,输入是一篇长文章,输出则是一个简短的摘要。

在训练阶段，Seq2Seq 模型采用监督学习的方式。给定一对输入-输出序列，模型通过通过最小化预测值与标签之间的交叉熵损失函数来学习参数。具体来说，在每个时间步，模型预测下一个字符的概率分布，并使用交叉熵损失函数来衡量预测与真实标签的差异。梯度通过反向传播在整个网络中流动，同时更新编码器和解码器的参数。

在推理或生成阶段，解码器自回归地生成输出：先生成第一个字符,然后将这个生成的字符作为输入来生成第二个字符，依此类推，直到生成终结符。这种自回归的特性导致生成过程中会累积错误，可能降低输出质量。

1.2 Attention

所谓注意力机制，就是让模型模拟人类观察一个场景时的处理模式：人脑并不会同时处理所有信息，而是会将注意力集中在关键区域上。这种选择性关注的能力使人类能够快速高效地处理复杂信息。在神经网络中，注意力机制是一种让模型能够动态关注输入不同部分的机制。它在每个输出步骤都计算一个“上下文向量”，这个向量是输入信息的加权组合，权重由当前任务的需求决定。

注意力机制的一般框架包含三个关键组件:

查询(Query)：表示当前需要关注什么。在生成任务中，查询通常是解码器当前时刻的隐状态，代表“我现在需要什么信息来生成下一个输出”。

键(Key)：用于与查询匹配，决定每个位置的重要性。在 Seq2Seq 中，键通常是编码器在各个时间步的输出，表示“输入的每个位置包含什么信息”。

值(Value)：实际被提取的信息。在大多数情况下，值和键相同，都是编码器的输出。

注意力机制通过计算查询和所有键之间的相关性,得到一组权重,然后用这些权重对值进行加权求和，得到最终的上下文向量。这个过程可以形式化为：

$$Attention(Q,K,V) = \sum softmax(Q \cdot K) \times V$$

其中 softmax 函数确保所有权重之和为 1，形成一个概率分布。这个分布表示了在生成当前输出时，输入序列每个位置的重要程度。

在基本的 Seq2Seq 模型中，编码器必须将整个输入序列的信息压缩到一个固定长度的向量中，这在序列较长时会导致信息丢失。注意力机制通过让解码器在生成每个输出时动态关注编码器的不同部分来解决这个问题。具体来说，在解码的每个时间步，模型会计算当前解码器状态(查询)与编码器所有时间步输出(键和值)之间的相关性，得到一组注意力权重。然后用这些权重对编码器输出进行加权求和，得到上下文向量。最后将上下文向量与解码器状态结合，用于预测下一个输出。以字符串逆置任务为例，当生成输出序列的第一个字符时，模型会重点关注输入序列的最后一个字符；生成第二个字符时，则关注倒数第二个字符，依此类推。

二. 实验过程与结果

2.1 代码补全

任务中，sequence_reversal-exercise.ipynb 的 call 函数和 sequence_reversal_with_attention-exercise.ipynb 的 call、get_next_token 函数代码需要自行补全，具体如下：

sequence_reversal-exercise.ipynb 的 call 函数：

```
def call(self, enc_ids, dec_ids):
        '''
        完成 sequence2sequence 模型的搭建，模块已经在`__init__`函数中定义好
        '''
        # Encoder 处理
        enc_emb = self.embed_layer(enc_ids)
        enc_out, enc_state = self.encoder(enc_emb)

        # Decoder 处理 - 使用 encoder 的最终状态作为初始状态
        dec_emb = self.embed_layer(dec_ids)
```

```
        dec_out, dec_state = self.decoder(dec_emb,
initial_state=enc_state)

        # 输出预测
        logits = self.dense(dec_out)
        return logits
```

sequence_reversal_with_attention-exercise.ipynb 的 call 函数：

```
def call(self, enc_ids, dec_ids):
        '''
        todo

        完成带 attention 机制的 sequence2sequence 模型的搭建，模块已经在
`__init__`函数中定义好，
        用双线性 attention，或者自己改一下`__init__`函数做加性 attention
        '''
        # Encoder 处理
        enc_emb = self.embed_layer(enc_ids)
        enc_out, enc_state = self.encoder(enc_emb)

        # Decoder 处理
        dec_emb = self.embed_layer(dec_ids)
        dec_out, dec_state = self.decoder(dec_emb,
initial_state=enc_state)

        # Attention 计算（双线性 attention）
        scores = tf.matmul(dec_out, enc_out, transpose_b=True)
        attn_weights = tf.nn.softmax(scores, axis=-1)
        context = tf.matmul(attn_weights, enc_out)

        # 结合 context 和 decoder 输出
        combined = tf.concat([dec_out, context], axis=-1)
        logits = self.dense(combined)
        return logits
```

sequence_reversal_with_attention-exercise.ipynb 的 get_next_token 函数：

```
def get_next_token(self, x, state, enc_out):
        '''
        shape(x) = [b_sz,]
        '''

        '''
        todo
        参考 sequence_reversal-exercise，自己构建单步解码逻辑'''
```

```
        # 嵌入
        inp_emb = self.embed_layer(x)
        inp_emb = tf.expand_dims(inp_emb, axis=1)

        # Decoder 一步
        h, state = self.decoder_cell.call(inp_emb, state)
        h = tf.squeeze(h, axis=1)

        # Attention
        scores = tf.matmul(h, enc_out, transpose_b=True)
        attn_weights = tf.nn.softmax(scores, axis=-1)
        context = tf.matmul(attn_weights, enc_out)

        # 结合
        combined = tf.concat([h, context], axis=-1)
        logits = self.dense(combined)

        # 选择下一个 token
        out = tf.argmax(logits, axis=-1)
        return out, state
```

此外，源代码还存在 Keras 版本与现有版本不兼容、训练前变量未初始化的问题，通过简单修改可以解决。

2.2 实验结果

训练和测试通过在 Jupiter 中运行 ipynb 文件进行。

普通 Seq2Seq 版本：

训练迭代过程：

```
step 0 : loss 3.3050466
step 500 : loss 1.5613896
step 1000 : loss 0.9045208
step 1500 : loss 0.6319083
step 2000 : loss 0.543289
step 2500 : loss 0.37690777
```

模型总共训练 3000 轮，每 500 轮打印一次 loss。可以看到 loss 稳定下降，收敛良好。

测试结果：

```
[True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True,
True, True, True, False, True, True, True]
[('SFJCFYDFUZ', 'ZUFDYFCJFS'), ('RMPPIOASVG', 'GVSAOIPPMR'), ('GDTTGINKPC', 'CPKNIGTTDG'), ('PZSQQNAPXH', 'HXPANQQSZP'), ('SZXCBLDBLS', 'SLBDLBCXZS'),
('GONPIQXGKE', 'EKGXQIPNOG'), ('BEAKRGYJPQ', 'QPJYGRKAEB'), ('CSTKCQCEID', 'DIECQCKTSC'), ('VQRMMBTCLU', 'ULCTBMMRQV'), ('VISDGNJGSV', 'VSGJNGDSIV'),
('SBKNDWYTNV', 'VNTYWDNKBS'), ('BPKIWNVGJQ', 'QJGVNWIKPB'), ('DAPAZULBPK', 'KPBLUZAPAD'), ('PEJYQVTCNQ', 'QNCTVQYJEP'), ('OOONMZXLKL', 'LKLXZMNOOO'),
('LYWCQUCKMQ', 'QMKCUQCWYL'), ('ADRLGRETAL', 'LATERGLRDA'), ('BLPZOQLIQI', 'IQILQOZPLB'), ('ZJEOUJIXWE', 'EWXIJUOEJZ'), ('ACHKBINEZK', 'KZENIBKHCA'),
('CVETDEZRMS', 'SMRZEDTEVC'), ('ZRZKPLATGJ', 'JGTALPKZRZ'), ('ZMNRBYBLEI', 'IELBYBRNMZ'), ('NEDHSTOQWA', 'AWQOTSHDEN'), ('PVGNYEHCJU', 'UJCHEYNGVP'),
('QKFKFNYYYT', 'TYYYNFKFKQ'), ('BWTFZCWQSU', 'USQWCZFTWB'), ('BYSDTWGYZS', 'SZYGWTDSYB'), ('TBYOXVPSNB', 'BNSPVXOYBT'), ('VEPBZDLRZZ', 'ZZRLDZBPEV'),
('ZFDGPMJSOS', 'SOSJMPGDFZ'), ('UVXTZNYCTJ', 'JTCYNZTXVU')]
```

总共 32 组测试数据，其中 31 组 True，准确率 31/32 = 96.875%。

结合 Attention 版本：

训练迭代过程：

```
step 0 : loss 3.303318
step 500 : loss 1.8092648
step 1000 : loss 1.1532023
step 1500 : loss 0.7442223
```

模型总共训练 2000 轮，每 500 轮打印一次 loss。可以看到 loss 稳定下降，收敛良好，但不如普通版本收敛快。

测试结果：

```
[False, False, False, False, False, False, False, False, False, True, False, False, False, True, False, False, False, False, False, False, True, False,
False, False, False, True, False, False, False, False, False, True]
[('TGINBOWVJPYPXQLTPKYZ', 'HKAYVLQXPYPJVWOBNIGT'), ('XYHFMCUEBLGATXMMICUF', 'FACYMMXVAGLBEUCMFHYX'), ('FRUOEGFANZNDTNBPJCIZ', 'UTCISANTDNZNAFGEOURF'),
('ZZCFOAFFFUASTTSZVVPT', 'PPAVZSTTOAUFFFAOFCZZ'), ('BELPZXUGHKYWUOKATVMI', 'IMVTAKOUWYKHGUXZPLEB'), ('AEIQKOSELXEDJWOCTXBL', 'LBMTCOWJDEXLESOKQIEA'),
('VBWZCLCBBZTVNRGMSICW', 'WCEKEGTNRTZBBCLCZWBV'), ('VQWIOBWLSFNINCMGHVXC', 'VXGHMSCAIJFSLWBOIWQV'), ('MRVTGOSBAXDKVDHZOUCI', 'VTUSJXAFLAJDBOOTTVRM'),
('CUBTFLHTSCUCUBPCGEHO', 'OHEGCPBUCUCSTHLFTBUC'), ('BOPXQCDCQAGUGXSEYDXO', 'ZMYJESXGUGAQCDCQXPOB'), ('DWCCSWLISNQVZWMXWDHX', 'MNFWMSDWALFSILWSCCWD'),
('PDEDFIWFHDXDCJPBDRDD', 'BCCDXAUBDYDHFWIFDEDP'), ('HHDXVSBZFRQEENQAFEFF', 'FQZTUMBEEQRFZBSVXDHH'), ('UWQRELHRGGCACKIPALPI', 'IQLPOIKCACGGRHLERQWU'),
('UIMVPFRSWTKVYFZFNIHE', 'QHINFJBYVKTWSRFPVMIU'), ('DIJEEFLLCXGLAZELILMR', 'NGPXLRCNRLEWLLFEEJID'), ('KZQYGUXURLXHPSZGKUFY', 'ORSXHLYKEGLRXDUGYQZK'),
('UMUFZJYKDVAPKNPYPCUE', 'RITCYINKPAVDKYJZFUMU'), ('AOVRVVKPADLHFGEKPSBK', 'OASKKIGFHLDAPKVVRVOA'), ('JDPGNSYIMKTHIWNHMFSY', 'KPJDHNZIHTKMIYSNGPDJ'),
('RKZTLPZXHEDVNHFYKPFG', 'CFPSYFHFVDEHXZPLTZKR'), ('PVFVXLJUCOAUOHTAFTKU', 'FBRFWTHOUAOCUJLXVFVP'), ('VHQBHEEHSXSZJMSEWAFI', 'TBHDQGHJZSXSJEEZBQHV'),
('HXKTAONXSGSALFYZFYAF', 'CSFENARLCOGSXNWDLPXH'), ('TSLLQJQMGSEJLWNNHGKC', 'CPYHWNDLJESGMQJQLLST'), ('AOXXYCBPMELHDOGZVRAV', 'LARBZGOJHLEMPBCYXXOA'),
('SUMVIRRVYXNHNJHWTHOG', 'PGHVWHJNHNXYVRRIVMUS'), ('RRLVKTOOZGWUABKCRONI', 'BRWRCKBAUWGZOOTKVLRR'), ('GRRYKLNXPIURDJMUJKVV', 'VLKJUMJDRUIPXNLKYRRG'),
('EHINXSGPYMJPNYAXVKUR', 'TUKBHLINPJMYPGSXNIHE'), ('NHDXIOKIHMUTILZQZQFM', 'QNQDQCLBTUMTIAOIXDHN')]
```

总共 32 组测试数据，其中 4 组 True，准确率 4/32 = 12.5%。

注意到加入 Attention 后准确率远低于普通版本，和预期不符。分析可能由于以下几点原因：

1. 训练步数不足。该版本只训练了 2000 步，低于普通版。
2. 测试字串更长。该版本测试的是长度为 20 的字串，而普通版本长度是 10。
3. 模型更复杂。带 Attention 后，模型参数应当是增加了，需要更多训练才能收敛，这点通过训练过程中在前 2000 轮 loss 下降不如普通版快也可看出。而未充分收敛的模型性能自然不佳。